

CS633 Assignment Group Number 51

Khusal Nikam 251110043, Prafull Joshi 251110056
Pranjal 251110058, Sarthak Dumbre 251110064
Vishal Junjare 251110085

February 15, 2026

1 Code Description

1.1 Overview

The program implements a distance-based communication and computation pattern using MPI. Each process exchanges data with two neighbors located at distances $D1$ and $D2$, performs the appropriate computation, and iteratively updates its local buffers for T iterations. The runtime of the iterative phase is measured, and the maximum time across all processes is reported.

1.2 MPI Initialization

```
MPI_Init(&argc, &argv);  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Each process initializes MPI, determines its rank within the communicator, and obtains the total number of processes.

1.3 Input Parameters

```
int M      = atoi(argv[1]);  
int D1     = atoi(argv[2]);  
int D2     = atoi(argv[3]);  
int T      = atoi(argv[4]);  
int seed   = atoi(argv[5]);
```

The program accepts five command-line arguments:

- M : Number of elements per buffer
- $D1$: First communication distance
- $D2$: Second communication distance

- T : Number of iterations
- seed : Random seed

1.4 Buffer Allocation and Initialization

```
double *buffer_D1 = malloc(M * sizeof(double));
double *buffer_D2 = malloc(M * sizeof(double));

srand(seed);
for (int i = 0; i < M; i++) {
    buffer_D1[i] = (double)rand() * (rank + 1) / 10000.0;
    buffer_D2[i] = buffer_D1[i];
}
```

Each process allocates local buffers of size M and initializes them with pseudo-random values scaled by the rank.

1.5 Neighbor Computation

```
int upD1    = (rank + D1 < size) ? rank + D1 : -1;
int downD1  = (rank - D1 >= 0)   ? rank - D1 : -1;

int upD2    = (rank + D2 < size) ? rank + D2 : -1;
int downD2  = (rank - D2 >= 0)   ? rank - D2 : -1;
```

Each process determines valid communication partners at distances $D1$ and $D2$. If a neighbor does not exist, the value is set to -1 .

1.6 Two-Phase Parity-Based Communication Strategy

Since blocking `MPI_Send` and `MPI_Recv` are used, a naive implementation where all processes send first would result in deadlock. To prevent this, a parity-based two-phase communication strategy is implemented.

Parity is computed as:

```
int evenD1 = ((rank / D1) % 2 == 0);
int evenD2 = ((rank / D2) % 2 == 0);
```

Processes are divided into two groups:

- Even parity group
- Odd parity group

Communication proceeds in two logical phases:

1. Phase A:

- Even group sends
- Odd group receives

2. Phase B:

- Odd group sends
- Even group receives

This ensures that at any moment, each send has a matching receive, eliminating the possibility of deadlock while maintaining structured distance-based communication.

1.7 Main Iterative Phase

Timing begins before entering the loop:

```
double start = MPI_Wtime();
```

The following steps are repeated for T iterations.

Phase 1: Request Communication

```
MPI_Send(buffer_D1, M, MPI_DOUBLE, upD1,
         TAG_D1_REQ, MPI_COMM_WORLD);

MPI_Recv(recv_D1, M, MPI_DOUBLE, downD1,
         TAG_D1_REQ, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Processes exchange buffer data with neighbors at distance $D1$ and $D2$ using blocking send and receive operations.

Phase 2: Computation

```
res_D1[i] = recv_D1[i] * recv_D1[i];
res_D2[i] = log(recv_D2[i]);
```

Two transformations are applied:

- $D1$: square operation
- $D2$: logarithmic transformation

Phase 3: Send Back Results

```
MPI_Send(res_D1, M, MPI_DOUBLE, downD1,
         TAG_D1_RES, MPI_COMM_WORLD);
```

The transformed results are sent back to the originating process.

Phase 4: Buffer Update

```
buffer_D1[i] =  
(double)((unsigned long long)recv_D1[i] % 100000ULL);  
  
buffer_D2[i] = recv_D2[i] * 100000.0;
```

After receiving the processed data, local buffers are updated. This creates iterative data evolution across T steps.

1.8 Performance Measurement

After completion of the loop:

```
double local_time = MPI_Wtime() - start;  
MPI_Reduce(&local_time, &max_time, 1,  
           MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
```

Only the iterative communication-computation phase is timed. The maximum runtime across all processes is computed using `MPI_Reduce` with `MPI_MAX`. This represents total parallel execution time.

1.9 Final Global Reduction

```
MPI_Reduce(&local_max_D1, &global_max_D1, 1,  
           MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
```

Each process computes local maxima of its buffers. Global maxima are computed collectively and printed by rank 0.

1.10 Summary

The program demonstrates:

- Distance-based structured communication
- Deadlock-safe blocking message passing
- Iterative buffer updates
- Parallel runtime measurement

It stresses communication bandwidth and computation, making it suitable for performance analysis on distributed systems such as PARAM Rudra.

2 Code Compilation and Execution Instructions

This section describes how to compile the MPI program, execute the experiments using Slurm batch scripts, collect timing results, and reproduce the performance plot shown in this report.

Provided Files

The submission contains the following files:

- `src.c` — MPI C source code.
- `src1.<P>_M.<M>.sh` — Slurm batch scripts used to run experiments for each process count (P) and data size (M).
- `assign1_*.out` — output files generated by Slurm jobs.
- `timing_data.txt` — aggregated timing values used for plotting.
- `plot.py` — Python script that generates the execution-time boxplot.

Compilation

The program is written using MPI and should be compiled using the MPI compiler wrapper:

```
mpicc -o src src.c -lm
```

The executable produced is named `src`.

Program Arguments and Output

The executable requires five command-line arguments:

```
./src <M> <D1> <D2> <T> <SEED>
```

where:

- M — data size (number of elements per process),
- $D1$ — first communication distance,
- $D2$ — second communication distance,
- T — number of iterations,
- $SEED$ — random seed.

At the end of execution, rank 0 prints:

```
<global_max_D1> <global_max_D2> <max_time>
```

where `max_time` is the maximum runtime across all MPI processes (obtained using `MPI_Reduce`) and is used as the performance metric in this report.

Interactive Execution

The code can be executed interactively using:

```
mpirun -np <P> ./src <M> <D1> <D2> <T> <SEED>
```

Example:

```
mpirun -np 16 ./src 262144 2 4 10 1000
```

Batch Execution Using SLURM

We have used SLURM batch scripts for each (P,M) configuration. Each script follows the same structure and runs 5 repeated executions.

Example :

```
#!/bin/bash
#SBATCH --job-name=mpi_assign1
#SBATCH -N 2
#SBATCH --ntasks-per-node=16
#SBATCH --output=assign1_16_M_262144_%j.out
#SBATCH --partition=cpu
#SBATCH --time=00:05:00

module load compiler/oneapi-2024/mpi

D1=2
D2=4
T=10
SEED=1000

for run in {1..5}
do
    mpirun -np 16 ./src 262144 $D1 $D2 $T $SEED
done
```

Key points:

- Two nodes are requested (`-N 2`) with 16 tasks per node.
- Output is written to files named: `assign1-<P>_M-<M>_<jobid>.out`.

The experiments include:

- Process counts: $P = 8, 16, 32$
- Data sizes: $M = 262144, 1048576$

Running the Experiments

Submit jobs using:

```
sbatch src1_8_M_262144.sh
sbatch src1_8_M_1048576.sh
sbatch src1_16_M_262144.sh
sbatch src1_16_M_1048576.sh
sbatch src1_32_M_262144.sh
sbatch src1_32_M_1048576.sh
```

Each job produces five timing measurements printed in the corresponding output file.

Typical Workflow Summary

```
#Load MPI module
module load compiler/oneapi-2024/mpi

# Compile
mpicc -o src src.c -lm

# Submit batch jobs
sbatch src1_8_M_262144.sh
sbatch src1_8_M_1048576.sh
sbatch src1_16_M_262144.sh
sbatch src1_16_M_1048576.sh
sbatch src1_32_M_262144.sh
sbatch src1_32_M_1048576.sh

# View job outputs
cat assign1_<P>_M_<M>_<jobid>.out

# Generate plot
python3 plot.py timing_data.txt
```

3 Results

3.1 Performance Results

Experiments were run using process counts $P = 8, 16$, and 32 , and two data sizes, $M = 262144$ and $M = 1048576$. Each setup was executed five times and the total execution time was recorded.

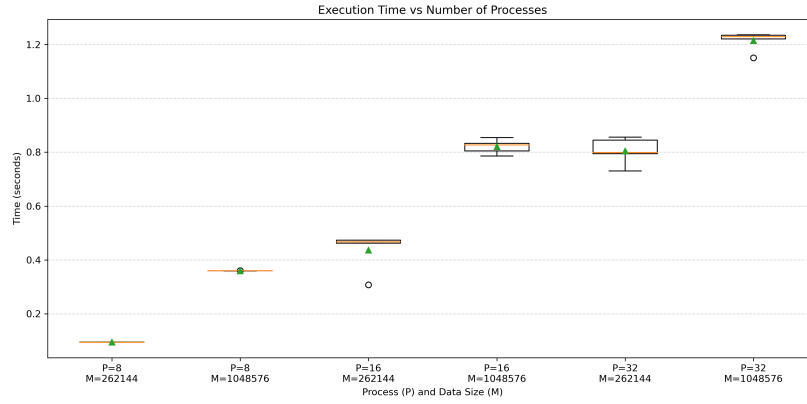


Figure 1: Execution time (seconds) for $M = 262144$ and $M = 1048576$ across $P = 8, 16, 32$. Each box shows five runs; the center line is the median, the box represents the interquartile range, and whiskers indicate the minimum and maximum values.

Figure 1 shows the execution time distribution for all tested configurations. The x-axis lists the combinations of process count and data size, while the y-axis shows execution time in seconds. The boxplots give a simple view of how the results vary across repeated runs.

Processes (P)	$M = 262144$	$M = 1048576$
8	0.09455	0.35987
16	0.43670	0.82105
32	0.80485	1.21441

Table 1: Average execution time (seconds) over five runs for each configuration.

Table 1 shows the average execution time measured for each experiment. These values provide a numerical summary of the results shown in the above plot.

3.2 Analysis of Results

The average execution times for $p = 8, 16$, and 32 processes for both data sizes are presented in Table 1 and Figure 1. The results clearly demonstrate the impact of communication overhead and scalability limitations of the implementation.

Effect of Increasing Number of Processes

For both data sizes, execution time increases as the number of processes increases.

Instead of decreasing with more processes, runtime increases significantly. This behavior indicates that the program does not exhibit strong scaling. Since each process handles a fixed workload of size M , increasing P increases total computation and communication volume.

Impact of Data Size

When the data size increases from $M = 262144$ to $M = 1048576$ (i.e., $4\times$ increase), the runtime also increases approximately by a factor of 3–4. This confirms that both computation and communication scale linearly with M .

For example, at $P = 8$:

$$0.35987/0.09455 \approx 3.8$$

This near-linear growth indicates predictable workload scaling.

Communication Overhead Analysis

The increase in runtime with higher P suggests that the program is communication-bound. Each iteration involves multiple blocking send and receive operations for buffers of size M . As the number of processes increases:

- Total communication volume increases.
- Network contention increases.
- Synchronization overhead increases.

At $P = 32$, communication cost dominates the overall runtime, resulting in the highest observed execution time.

Performance Efficiency Evaluation

Positive aspects:

- Balanced workload across processes.
- Deadlock-free communication using parity strategy.

- Near-linear scaling with data size.

Limitations:

- Work per process remains constant, so increasing P increases total work.
- Blocking communication prevents overlap of computation and communication.
- Communication overhead dominates at larger process counts.

Overall Observation

The results indicate that the implementation scales well with problem size but does not scale efficiently with increasing number of processes. The primary limitation arises from communication overhead and synchronization costs, which become significant at higher process counts.

4 Conclusions

This work implements a distance-based MPI communication program using a parity-based method to avoid deadlocks while using blocking communication. From the results, it is observed that execution time increases when either the data size or the number of processes increases. The runtime grows almost proportionally with larger data sizes, which shows that the workload scales in a predictable way. However, when the number of processes increases, the execution time also increases because of higher communication and synchronization overhead between processes.

The results indicate that the program is mainly limited by communication cost rather than computation. Since blocking communication is used, processes spend more time waiting as the process count grows, which affects scalability. Overall, the implementation works correctly and shows consistent performance trends, while clearly demonstrating how communication overhead impacts parallel performance.

All team members have contributed equally to the design, implementation, experimentation, and preparation of this assignment report.